SAÉ S1.01 : Méthodologie d'Implémentation d'un Besoin Client et Conception Algorithmique

Thème principal : Conception logicielle, modélisation fonctionnelle et tests unitaires

Synthèse exécutive : La ressource S1.01 constitue le premier projet intégrateur du BUT Informatique. Son enjeu fondamental est de combler le fossé entre une demande client exprimée en langage naturel et une implémentation logicielle rigoureuse, modulaire et vérifiable. Ce cours pose le cadre méthodologique complet : analyse du cahier des charges, modélisation fonctionnelle, décomposition descendante d'un problème algorithmique, contractualisation logicielle et validation systématique par tests unitaires.

De l'Expression du Besoin à la Spécification Fonctionnelle

En informatique professionnelle, un projet échoue rarement à cause d'une incapacité à coder, mais presque toujours à cause d'un décalage entre ce que le client a exprimé, ce que le développeur a compris, et ce qui a été testé. L'objectif initial consiste donc à transformer un énoncé informel en un ensemble de règles explicites. Chaque fonctionnalité doit répondre au triptyque 'Étant donné / Quand / Alors'. Toute ambiguïté lexicale doit être bannie au profit de seuils quantifiables.

- Expression de besoin : formulation subjective et parfois contradictoire fournie par l'utilisateur final
- Spécification fonctionnelle : traduction technique univoque décrivant le comportement attendu du système sans présager de l'implémentation
- Critères d'acceptation : conditions observables permettant d'arbitrer si une fonctionnalité est valide

Cas concret enseignant : *L'enseignant illustre ce problème avec l'analogie de la 'balançoire d'entreprise' : le client demande une balançoire suspendue à une branche, mais chaque corps de métier l'interprète différemment jusqu'à couper le tronc de l'arbre en deux pour faire passer la corde, alors que le client voulait simplement un pneu attaché avec un câble.*

Décomposition Modulaire et Conception Descendante

Pour concevoir un programme robuste, l'étudiant doit proscrire la rédaction d'un bloc de code monolithique dans une seule fonction 'main'. La démarche requise est la décomposition descendante. Un problème global P est découpé en sous-tâches P1, P2, ..., Pn. Chaque sous-tâche se matérialise par une fonction dotée d'une responsabilité unique (Single Responsibility Principle). On sépare systématiquement la logique métier de la couche d'interface utilisateur.

- Conception descendante (Top-Down) : démarche consistant à scinder un problème complexe en sous-problèmes élémentaires indépendants
- Modularité : découpage du code en modules autonomes présentant une forte cohésion interne et un faible couplage
- Signature de fonction : contrat liant l'identifiant de la fonction, la nature de ses arguments et son type de retour

Cas concret enseignant : *Métaphore de la chaîne de montage automobile : le technicien qui assemble l'alternateur n'a pas besoin de savoir quelle est la couleur des sièges en cuir. Si une fonction de calcul fiscal contient des instructions d'affichage à l'écran, elle devient inutilisable sur un serveur web ou dans une application mobile.*

Programmation par Contrat et Stratégie de Test

Une fonction mathématique ou informatique n'est fiable que si son domaine de validité est formellement délimité. La programmation par contrat stipule que si l'appelant respecte les préconditions, la fonction s'engage à satisfaire les postconditions. Les assertions permettent de stopper immédiatement l'exécution lors de la phase de débogage si une précondition est violée.

- Précondition : hypothèse indispensable sur les paramètres d'entrée pour garantir un résultat correct

• Postcondition : état du système et valeur de retour garantis par la fonction à la fin de son exécution

• Test unitaire : fonction automatisée vérifiant qu'une unité de code réagit conformément aux spécifications

Cas concret enseignant : *Le professeur insiste sur l'accident de la fusée Ariane 5 (vol 501 en 1996) : l'explosion a été provoquée par une conversion d'un nombre flottant 64 bits en entier 16 bits sans vérification des préconditions sur la plage de valeurs admissibles.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne peut-on pas mettre de 'printf' ou d'affichages dans nos fonctions de calcul d'algorithmes ?
Réponse du professeur : Parce que cela crée un couplage fort entre le calcul et l'interface utilisateur. Si vous voulez réutiliser votre fonction dans une application graphique ou web, l'affichage console plantera ou ne servira à rien.

Question étudiant : Quelle est la différence exacte entre une précondition et une condition de validation de saisie utilisateur ?
Réponse du professeur : La validation de saisie gère l'erreur humaine au clavier (boucle de redemandage). La précondition est un contrat interne entre développeurs : si elle est violée, c'est qu'il y a un bug dans le code appelant, d'où l'usage d'un 'assert'.

■ Exercice & Démonstration d'Application

Énoncé : Spécifier et implémenter une fonction modulaire en C/Python 'calculer_moyenne_ponderee' qui prend en entrée une liste de notes (flottants sur 20) et une liste de coefficients entiers correspondants, puis renvoie la moyenne pondérée.

Résolution pas à pas :
Étape 1 (Spécification et Contrats) : - Préconditions : Les deux listes doivent avoir la même taille N > 0. Chaque note doit être comprise entre 0.0 et 20.0. Chaque coefficient doit être un entier strictement supérieur à 0. - Postcondition : Renvoie un flottant compris entre 0.0 et 20.0. Étape 2 (Algorithme) : 1. Initialiser 'somme_points' à 0.0 et 'somme_coeffs' à 0. 2. Parcourir les indices de 0 à N-1 : - Multiplier la note par le coefficient. - Ajouter à 'somme_points' et 'somme_coeffs'. 3. Retourner 'somme_points / somme_coeffs'. Étape 3 (Cas limites) : - Test nominal : notes [10, 14], coeffs [1, 3] -> (10*1 + 14*3)/4 = 13.0.

Résultat : La fonction est découplée de toute entrée/sortie console. Elle garantit l'intégrité arithmétique et peut être validée instantanément par une batterie de tests unitaires.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le risque majeur d'un couplage fort entre le traitement algorithmique et l'affichage console ?

R : L'impossibilité de réutiliser la fonction dans un autre contexte (Web, GUI) et la difficulté d'automatiser les tests unitaires.

Q : Qu'appelle-t-on un cas limite (edge case) en conception algorithmique ?

R : Une situation critique aux bornes du domaine de validité (listes vides, valeurs minimales/maximales, zéros) qui déclenche souvent des bugs si elle n'est pas explicitement traitée.

Q : À quoi sert une assertion (assert) dans un sous-programme ?

R : À vérifier formellement qu'une précondition est respectée à l'exécution et à interrompre immédiatement le programme en cas d'anomalie de développement.